HONY LINK

API Reference

GraphQL — Backend & Frontend Integration Guide

Base URL	http://localhost:4000/graphql
Protocol	GraphQL over HTTP POST
Auth	JWT Bearer Token (Authorization header)
Token Store	expo-secure-store (mobile) / SecureStore
Version	1.0.0 · March 2026

This document covers every GraphQL query and mutation exposed by the Hony Link API, including full parameter tables, example request/response payloads, and the corresponding Apollo client calls used in the Expo mobile app.

1. Authentication

All protected operations require a valid JWT passed in the Authorization header. Tokens are issued on signup or login and expire after 7 days.

```
# HTTP Header required for protected operations
Authorization: Bearer <your_jwt_token>
```

1.1 signup — Create a new account

[MUTATION]

Registers a new user, creates a blank profile and a free subscription, and returns an auth token.

```
mutation Signup($email: String!, $password: String!, $username: String!) {
  signup(email: $email, password: $password, username: $username) {
    token
    user {
      id  username  email  isVerified
      profile { bio age city }
      subscription { plan status }
    }
  }
}
```

Parameter	Type	Required	Description
email	String!	✓	User's email address — must be unique
password	String!	✓	Plain-text password (hashed with bcrypt)
username	String!	✓	Display name — must be unique

Field	Type	Description
token	String!	JWT for subsequent authenticated requests
user.id	ID!	MongoDB ObjectId
user.username	String!	Chosen display name
user.profile	Profile	Initially blank profile object
user.subscription	Subscription	Defaults to free plan

■ *Apollo call: useMutation(SIGNUP) → await login(data.signup.token, data.signup.user)*

1.2 login — Sign in to existing account

[MUTATION]

Authenticates credentials and returns a fresh JWT.

```
mutation Login($email: String!, $password: String!) {
  login(email: $email, password: $password) {
    token
    user { id username email profile { bio age city photos } }
  }
}
```

Parameter	Type	Required	Description

email	String!	✓	Registered email address
password	String!	✓	Account password

■ *On success, store token with: await SecureStore.setItemAsync('honylink_token', token)*

2. User & Profile

2.1 me — Fetch the authenticated user

[QUERY]

Returns the full profile of the currently authenticated user.

```
query Me {
  me {
    id  username  email  isVerified
    profile {
      bio  age  gender  city  photos  interests
      lookingFor  minAge  maxAge
    }
    subscription { plan  status  endDate }
  }
}
```

Field	Type	Description
id	ID!	User's unique identifier
profile.photos	[String]	Array of image URLs
profile.interests	[String]	Array of interest tags
profile.location	Point	GeoJSON coordinates (not exposed in GraphQL — set via updateProfile)
subscription.plan	String	free | basic | premium
subscription.endDate	Date	ISO date string — null for free plan

2.2 updateProfile — Edit profile fields

[MUTATION]

Updates any combination of profile fields. All fields are optional.

```
mutation UpdateProfile($input: UpdateProfileInput!) {
  updateProfile(input: $input) {
    bio  age  gender  city  photos  interests
    lookingFor  minAge  maxAge  updatedAt
  }
}

# Example variables
{
  "input": {
    "bio": "Love hiking and coffee",
    "age": 27,
    "city": "Lagos",
    "interests": ["hiking", "coffee", "travel"],
    "latitude": 6.5244,
    "longitude": 3.3792
  }
}
```

Parameter	Type	Required	Description
bio	String	—	Short personal bio (max 500 chars)
age	Int	—	User age (18–100)
gender	String	—	male | female | non-binary | other
city	String	—	City name for display and search
photos	[String]	—	Array of image URLs
interests	[String]	—	Interest tags used in smart matching
latitude	Float	—	GPS latitude — stored as 2dsphere point
longitude	Float	—	GPS longitude — stored as 2dsphere point
lookingFor	String	—	Preferred gender: male | female | any | ...
minAge	Int	—	Minimum preferred partner age
maxAge	Int	—	Maximum preferred partner age

3. Swipe & Matching

The swipe system is the core of Hony Link. A match is created only when both users swipe RIGHT on each other.

3.1 swipeFeed — Get ranked candidates

[QUERY]

Returns a list of users the current user has not yet swiped, ranked by the smart matching algorithm (shared interests 40 pts + age compatibility 30 pts + distance 30 pts).

```
query SwipeFeed($limit: Int) {
  swipeFeed(limit: $limit) {
    id  username
    profile { bio  age  gender  city  photos  interests }
  }
}
```

Parameter	Type	Required	Description
limit	Int	—	Max users to return. Default: 20

3.2 swipe — Swipe left or right

[MUTATION]

Records a swipe action. If the target user has already swiped RIGHT on the current user, a Match is created and returned.

```
mutation Swipe($toUserId: ID!, $direction: String!) {
  swipe(toUserId: $toUserId, direction: $direction) {
    direction
    matched
    match {
      id  createdAt
      users { id  username  profile { photos } }
    }
  }
}
```

Parameter	Type	Required	Description
toUserId	ID!	✓	ID of the user being swiped on
direction	String!	✓	LEFT or RIGHT (case-insensitive)

Field	Type	Description
direction	String	The recorded direction (LEFT | RIGHT)
matched	Boolean	True when a mutual RIGHT swipe creates a match

match	Match	Populated only when matched = true

■ *In the Expo app, the SwipeScreen detects matched: true and shows a match banner.*

3.3 myMatches — List all matches

[QUERY]

```
query MyMatches {
  myMatches {
    id  createdAt
    users { id  username  profile { photos  city } }
  }
}
```

Field	Type	Description
id	ID!	Match ID
users	[User!]	Both participants in the match
createdAt	Date	ISO timestamp of when the match was made

3.4 searchUsers — Search by username / location / age

[QUERY]

```
query SearchUsers(
  $q: String!  $limit: Int
  $city: String  $minAge: Int  $maxAge: Int
) {
  searchUsers(q: $q, limit: $limit, city: $city,
              minAge: $minAge, maxAge: $maxAge) {
    id  username
    profile { bio  age  gender  city  photos  interests }
  }
}
```

Parameter	Type	Required	Description
q	String!	✓	Username search term (regex, case-insensitive)
limit	Int	—	Max results. Default: 20
city	String	—	Filter by city name
minAge	Int	—	Minimum age filter

maxAge	Int	—	Maximum age filter

4. Chat & Messaging

Free users are limited to 5 new chat conversations per day. Existing conversations remain accessible regardless of limit.

4.1 createChat — Start a new conversation

[MUTATION]

Opens a chat between the current user and another user. If a chat already exists, returns the existing one.

```
mutation CreateChat($userId: ID!) {
  createChat(userId: $userId) {
    id  createdAt
    participants { id  username  profile { photos } }
  }
}
```

Parameter	Type	Required	Description
userId	ID!	✓	ID of the other participant

■ *Throws: 'Free users can start at most 5 new chats per day. Upgrade to continue.'*

4.2 chatList — Fetch all conversations

[QUERY]

Returns all chats the current user is part of, sorted by most recent activity. The Expo app polls this every 10 seconds.

```
query ChatList($limit: Int) {
  chatList(limit: $limit) {
    id  updatedAt
    participants { id  username  profile { photos } }
    lastMessage  { text  createdAt  sender { username } }
  }
}
```

4.3 messages — Fetch messages in a chat

[QUERY]

Returns the most recent messages in a chat, newest first. The Expo app polls this every 3 seconds to simulate real-time.

```
query Messages($chatId: ID!, $limit: Int) {
  messages(chatId: $chatId, limit: $limit) {
    id  text  createdAt
    sender { id  username  profile { photos } }
  }
}
```

Parameter	Type	Required	Description
chatId	ID!	✓	ID of the chat conversation
limit	Int	—	Number of messages to return. Default: 30

4.4 sendMessage — Send a message

[MUTATION]

```
mutation SendMessage($chatId: ID!, $text: String!) {
  sendMessage(chatId: $chatId, text: $text) {
    id  text  createdAt
    sender { id  username  profile { photos } }
  }
}
```

Parameter	Type	Required	Description
chatId	ID!	✓	Target chat conversation
text	String!	✓	Message content (max 500 chars)

■ *Also updates Chat.lastMessage and Chat.updatedAt server-side.*

5. Short Videos

Short video content is gated behind a paid subscription. Free users see an upgrade prompt instead of the feed.

5.1 shortsFeed — Get video feed

[QUERY]

Returns the most recent short videos, newest first. Throws an error if the user is on the free plan.

```
query ShortsFeed($limit: Int) {
  shortsFeed(limit: $limit) {
    id  videoUrl  caption  likesCount  createdAt
    uploader { id  username  profile { photos } }
  }
}
```

■ *Throws: 'Shorts are available to subscribers only. Upgrade your plan to watch videos.'*

5.2 uploadShort — Upload a video

[MUTATION]

```
mutation UploadShort($videoUrl: String!, $caption: String) {
  uploadShort(videoUrl: $videoUrl, caption: $caption) {
    id  videoUrl  caption  createdAt
    uploader { id  username }
  }
}
```

Parameter	Type	Required	Description
`videoUrl`	`String!`	✓	URL of the uploaded video file
`caption`	`String`	—	Optional caption (max 300 chars)

6. Subscription Plans

Upgrading a plan unlocks shorts, removes daily chat limits, and improves swipe feed ranking. Plans are billed monthly.

Plan	Price	Chats/day	Shorts	Matching
free	■0	5	✗	Standard
basic	■2,499	Unlimited	✓	Standard
premium	■4,999	Unlimited	✓	Priority

6.1 subscribe — Change subscription plan

[MUTATION]

```
mutation Subscribe($plan: String!) {
  subscribe(plan: $plan) {
    id  plan  status  startDate  endDate
  }
}

# Variables: { "plan": "premium" }
```

Parameter	Type	Required	Description
plan	String!	✓	free | basic | premium

Field	Type	Description
plan	String	Active plan name
status	String	active | cancelled | expired
startDate	Date	Start of current billing period
endDate	Date	End of billing period (null for free)

7. Error Handling

All errors are returned in the standard GraphQL errors array. The Expo app's Apollo error link logs these automatically.

```
# GraphQL error response shape
{
  "data": null,
  "errors": [
    {
      "message": "Authentication required. Please log in.",
      "locations": [{ "line": 2, "column": 3 }],
      "path": ["me"]
    }
  ]
}
```

Error message	Cause
Authentication required.	No / expired JWT
Invalid email or password.	Wrong login credentials
Email or username already in use.	Duplicate signup

`Access denied.`	Not a chat participant
`Free users can start at most 5…`	Daily chat limit hit
`Shorts are available to subscribers…`	Free user accessing shorts
`OPay error 02001: …`	Missing callbackUrl in payment

8. Frontend Integration Summary

The Expo app in src/apollo/client.js uses Apollo Client with three links chained together: errorLink → authLink → httpLink. The authLink reads the JWT from SecureStore before every request.

```
# Apollo link chain (src/apollo/client.js)
from([errorLink, authLink, httpLink])

# authLink — attaches JWT
const authLink = setContext(async (_, { headers }) => {
  const token = await SecureStore.getItemAsync('honylink_token');
  return { headers: { ...headers,
    authorization: token ? `Bearer ${token}` : '' } };
});
```

Screen	Operation	Hook	Polling
Login	`login/signup`	useMutation	—
Swipe	`swipeFeed`	useQuery	on refetch
Swipe	`swipe`	useMutation	—
Matches	`myMatches`	useQuery	—
Matches	`createChat`	useMutation	—
Chat List	`chatList`	useQuery	10 s
Chat Room	`messages`	useQuery	3 s
Chat Room	`sendMessage`	useMutation	—
Shorts	`shortsFeed`	useQuery	—
Profile	`me`	useQuery	on refetch
Profile	`updateProfile`	useMutation	—
Profile	`subscribe`	useMutation	—